AI COURSE – TASK 1

Understand & Visualize: Convolutional Neural Network (CNN)

Architecture chosen	Convolutional Neural Network (CNN)
Difficulty	Basic
Practical model	Pretrained ResNet18 (CNN)

1. Introduction to CNN

A Convolutional Neural Network (CNN) is a type of artificial neural network mainly used to understand images. It is commonly used in image classification, face recognition, medical image analysis and object detection. A computer receives an image as a group of pixel values. A CNN learns useful patterns from these pixels and uses them to identify what is present in the image.

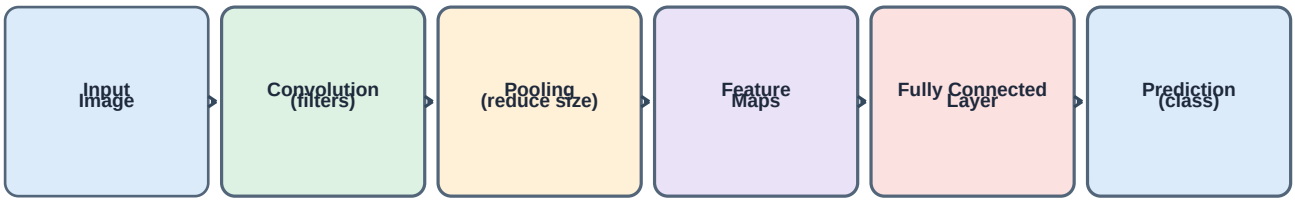
A CNN usually has several important layers. The input layer receives the image. The convolution layer uses small filters, also called kernels, to scan the image and find features such as edges, lines, corners and textures. Deeper layers can learn more complex features such as eyes, wheels, leaves or shapes. The output of a convolution layer is called a feature map.

A pooling layer reduces the size of feature maps. This makes processing faster and helps the model focus on important information. After feature extraction, the fully connected layer combines the learned features and makes a decision. The final output layer gives a prediction, such as cat, dog, car or bird. During training, the CNN compares its prediction with the correct answer and adjusts its internal weights to reduce errors.

One major advantage of CNNs is that they automatically learn image features instead of requiring a person to define every feature manually. CNNs can also recognise patterns when an object appears in slightly different positions. However, training a CNN from the beginning may require many labelled images, powerful hardware and considerable time.

For this assignment, a pretrained ResNet18 model is used. ResNet18 is a CNN that has already been trained on a large image dataset. We use it to predict the categories of five sample images. This demonstrates inference, which means using a trained model to make predictions on new inputs.

2. Labelled CNN Architecture Diagram



A CNN learns image features and uses them to predict the image category.

Diagram explanation: The image passes through convolution and pooling operations. The network extracts useful features, the fully connected layer uses those features, and the output gives the predicted class.

3. Practical Implementation – Pretrained CNN

Model: ResNet18 with pretrained ImageNet weights. The program reads five image files, prepares them for the model, predicts the most likely class, and prints the result.

Before running the program

Use Python 3.11 for better compatibility with PyTorch. Open Command Prompt and install the required libraries:

```
py -3.11 -m pip install torch torchvision pillow
```

Create a folder named **sample_images** beside the Python file. Put five JPG or PNG images inside it and name them image1.jpg, image2.jpg, image3.jpg, image4.jpg and image5.jpg.

```
import torch
from torchvision.models import resnet18, ResNet18_Weights
from PIL import Image

# Load a pretrained CNN model
weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)
model.eval()

preprocess = weights.transforms()
categories = weights.meta["categories"]

image_files = [
    "sample_images/image1.jpg",
    "sample_images/image2.jpg",
    "sample_images/image3.jpg",
    "sample_images/image4.jpg",
    "sample_images/image5.jpg"
]

# Predict the class of each image
for file_name in image_files:
    try:
        image = Image.open(file_name).convert("RGB")
        input_tensor = preprocess(image).unsqueeze(0)

        with torch.no_grad():
            output = model(input_tensor)
            probabilities = torch.nn.functional.softmax(output[0], dim=0)
            score, index = torch.max(probabilities, dim=0)

        print(file_name, "->", categories[index.item()],
              "| confidence:", round(score.item() * 100, 2), "%")

    except FileNotFoundError:
        print("Image not found:", file_name)
```

How to run: Save the code as **inference.py**. Keep the sample_images folder beside it. Run **py -3.11 inference.py** in Command Prompt. The first run downloads the pretrained model weights, so an internet connection is needed.

4. Five Sample Inputs and Outputs

The table below gives examples of the type of output the program may produce. Actual predictions and confidence scores depend on the images you choose. Copy the results from your own program run.

Sample input	Example predicted output
image1.jpg (dog photo)	Dog category
image2.jpg (cat photo)	Cat category
image3.jpg (car photo)	Car category
image4.jpg (bird photo)	Bird category
image5.jpg (flower photo)	Flower category

5. Result and Conclusion

The assignment demonstrates how a Convolutional Neural Network processes images and predicts their categories. The pretrained ResNet18 model can classify new images without training a model from the beginning. The five predictions show a basic example of AI-based image recognition. A prediction is not always correct because image quality, angle, lighting and similarity between objects can affect the result.

SUBMISSION CHECKLIST

Before submitting Task 1, make sure you have completed each item below.

■ **README.md:** Write a 300–400 word explanation of CNN in your own words.

■ **Labelled diagram:** Add the CNN architecture diagram to your README.md.

■ **Python program:** Add the inference.py file using the pretrained ResNet18 model.

■ **Five sample images:** Run the program using five image inputs.

■ **Actual outputs:** Include the five predictions and brief comments on the results.

■ **GitHub upload:** Upload README.md, inference.py and required supporting files to your repository.

■ **Repository access:** Check that your GitHub repository is accessible to your instructor.

■ **LMS submission:** Copy your GitHub repository URL and paste it into the Task 1 submission field in the Student LMS.

Final reminder: Replace the example outputs in the assignment with the actual results from your program before submitting.